

**ĐẠI HỌC QUỐC GIA HÀ NỘI**  
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



# **XÂY DỰNG TRÒ CHƠI CỜ CARO VỚI THUẬT TOÁN AI MINIMAX CÙNG CẮT TỈA ALPHA-BETA**

**TRÍ TUỆ NHÂN TẠO**

**Giảng viên: PGS.TS Nguyễn Phương Thái**

**Sinh viên**

|                         |                 |
|-------------------------|-----------------|
| <b>Lê Khánh Toàn</b>    | <b>23020308</b> |
| <b>Nguyễn Văn tiến</b>  | <b>23020307</b> |
| <b>Nguyễn Văn Quyển</b> | <b>23020306</b> |

**Hà Nội, 2026**

## Contents

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 1. Mô tả bài toán Cờ Caro .....                                                     | 4  |
| 2. Luật chơi và điều kiện thắng .....                                               | 4  |
| 3. Cách biểu diễn trạng thái bàn cờ .....                                           | 4  |
| 4. Cách sinh nước đi hợp lý .....                                                   | 4  |
| 4.1. Cơ chế tối ưu hóa .....                                                        | 4  |
| 4.2. Cơ chế khử các ô bị trùng lặp .....                                            | 5  |
| 5. Cách kiểm tra trạng thái kết thúc .....                                          | 5  |
| 6. Thuật toán Minimax đã cài đặt .....                                              | 6  |
| 6.1. Cơ chế Min và Max .....                                                        | 6  |
| 6.2. Xử lý trạng thái kết thúc và giới hạn độ sâu .....                             | 7  |
| 6.3. Cách thức AI tìm kiếm nước đi qua Depth .....                                  | 7  |
| 7. Thuật toán Alpha Beta Pruning đã cài đặt .....                                   | 8  |
| 8. Hàm đánh giá trạng thái .....                                                    | 9  |
| 8.1. Cơ chế quét ô .....                                                            | 9  |
| 8.2. Hệ thống đánh giá điểm .....                                                   | 10 |
| 8.3. Phân tích tư duy thông minh của AI qua hàm đánh giá .....                      | 10 |
| 9. Giao diện của game .....                                                         | 10 |
| 10. Bảng kết quả thực nghiệm và nhận xét .....                                      | 11 |
| 10.1. Trạng thái đầu ván .....                                                      | 11 |
| 10.2. Trạng thái giữa trận .....                                                    | 13 |
| 10.3. Trạng thái máy có thể thắng ngay .....                                        | 14 |
| 10.4. Trạng thái máy phải chặn thắng .....                                          | 15 |
| 10.5. Trạng thái máy phải lựa chọn giữa Thắng hoặc Chặn Thắng .....                 | 16 |
| 11. Đánh giá và thảo luận tổng quan .....                                           | 17 |
| 11.1. Alpha-Beta có chọn cùng nước đi với Minimax không? .....                      | 17 |
| 11.2. Alpha-Beta giảm được bao nhiêu trạng thái so với Minimax? .....               | 17 |
| 11.3. Thời gian chạy thay đổi như thế nào khi tăng độ sâu? .....                    | 17 |
| 11.4. Độ sâu tìm kiếm ảnh hưởng thế nào đến chất lượng nước đi? .....               | 17 |
| 11.5. Hàm đánh giá có ưu điểm gì và còn hạn chế gì? .....                           | 17 |
| 11.6. Trường hợp nào AI chơi tốt và trường hợp nào AI chọn nước đi chưa hợp lý? ... | 18 |
| 11.7. Nếu cải tiến tiếp, sinh viên sẽ cải tiến phần nào? .....                      | 18 |
| 12. Ưu điểm và nhược điểm .....                                                     | 18 |

|                                    |           |
|------------------------------------|-----------|
| <b>12.1. Ưu điểm .....</b>         | <b>18</b> |
| <b>12.2. Nhược điểm .....</b>      | <b>19</b> |
| <b>13. Tài liệu tham khảo.....</b> | <b>19</b> |

# 1. Mô tả bài toán Cờ Caro

Bài tập lớn yêu cầu xây dựng một chương trình Trí tuệ nhân tạo chơi cờ Caro giữa Người (đánh quân X) và Máy (đánh quân O). Máy sẽ được xây dựng thuật toán Minimax cùng với Alpha Beta Pruning để chọn ra nước đi tối ưu nhất.

## 2. Luật chơi và điều kiện thắng

Trò chơi Cờ Caro của em được làm với những thiết lập cơ bản sau:

- Bàn cờ: Kích thước 9x9 lưới ô vuông
- Cách thức chơi: Hai người chơi lần lượt đánh ô X và O vào những ô còn trống trên bàn cờ
- Điều kiện thắng: Một bên sẽ chiến thắng khi và chỉ khi đánh được 4 ô X hoặc O liên tiếp lên bàn cờ mà không bị đối thủ chặn đầu. Theo yêu cầu của đề bài, trò chơi này không có tích hợp luật Chặn hai đầu
- Điều kiện hòa: Nếu như 9x9 ô lưới vuông bị lấp đầy mà chưa tìm được người chiến thắng

## 3. Cách biểu diễn trạng thái bàn cờ

- Chương trình áp dụng dữ liệu logic cùng với giao diện đồ họa Pygame để thiết lập bàn cờ.
- Trạng thái của bàn cờ được lưu giữ dưới dạng một mảng hai chiều. Mảng này có kích thước 9x9 và mỗi phần tử `grid[r][c]` đại diện cho một ô cờ, chỉ nhận một trong ba giá trị sau:
  - + EMPTY = 0: Trạng thái ô trống
  - + PLAYER\_X = 1: Trạng thái của người chơi đánh X
  - + PLAYER\_Y = 1: Trạng thái của người chơi đánh Y

## 4. Cách sinh nước đi hợp lý

Vì trò chơi cờ Caro của bài toán có kích thước là 9x9 ô, việc xét tất cả các nước đi trống sẽ khiến cho máy tính phải hoạt động quá nhiều khiến tốc độ phản hồi rất chậm.

Để khắc phục vấn đề này, chương trình sử dụng Proximity Search để tối ưu hóa việc sinh nước đi

### 4.1. Cơ chế tối ưu hóa

Thay vì duyệt toàn bộ các ô EMPTY trên bàn cờ, thuật toán chỉ tìm và trả về các ô trống nằm liền kề dọc – ngang – chéo với các ô đã có quân cờ vì trong trò chơi cờ Caro, các nước đi có ý nghĩa thường nằm cạnh các ô đã đi vì điều kiện thắng là phải xếp được 4 ô liền kề nhau. Việc

đánh vào một ô không liên quan tới các nước đi có sẵn là vô nghĩa. Cắt bỏ được việc này giúp thuật toán AI giảm đáng kể trường hợp phải xét

## 4.2. Cơ chế khử các ô bị trùng lặp

Vì một ô trống có thể nằm lân cận với nhiều quân cờ khác nhau, nếu dùng List sẽ sinh ra các nước đi bị lặp lại. Do đó, chương trình sử dụng cấu trúc dữ liệu Set để tự động loại bỏ các tọa độ trùng lặp.

```
def generate_moves(board): 4 usages
    moves = set()
    for r in range(GRID_SIZE):
        for c in range(GRID_SIZE):
            if board[r][c] != EMPTY:
                for dr in [-1, 0, 1]:
                    for dc in [-1, 0, 1]:
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < GRID_SIZE and 0 <= nc < GRID_SIZE and board[nr][nc] == EMPTY:
                            moves.add((nr, nc))
    return list(moves)
```

## 5. Cách kiểm tra trạng thái kết thúc

Cơ chế hoạt động: Hệ thống xác định chiến thắng dựa trên quy tắc 4 quân liên tiếp mà không xét luật chặn 2 đầu. Để kiểm tra nhanh chóng và tối ưu, chương trình sử dụng thuật toán quét Sliding Window có kích thước cố định là 4 ô. Thuật toán sẽ quét qua mảng 2 chiều theo 4 hướng độc lập:

- Hướng ngang: Kiểm tra `board[r][c] == board[r][c+1] == board[r][c+2] == board[r][c+3]`.
- Hướng dọc: Kiểm tra `board[r][c] == board[r+1][c] == board[r+2][c] == board[r+3][c]`.
- Hướng chéo: Kiểm tra từ góc trên-trái xuống dưới-phải và kiểm tra từ góc trên-phải xuống dưới-trái

Sau mỗi nước đi của Người chơi hoặc Máy, hệ thống sẽ tự động gọi hàm `check_win_logic` để quét toàn bộ dữ liệu trên mảng hai chiều. Trạng thái kết thúc được xác định ngay khi một trong hai bên đạt đủ số quân liên tiếp theo quy định (4 quân) hoặc khi không còn ô trống nào để thực hiện nước đi.

Trong trường hợp hàm đã quét hết bàn cờ mà không tìm thấy chuỗi thắng nào, đồng thời kiểm tra thấy tất cả các phần tử trong mảng logic đều đã được lấp đầy (không còn ô trống EMPTY nào), trận đấu sẽ được tuyên bố kết thúc với kết quả Hòa.

```
def check_win_logic(board): 4 usages
    for r in range(GRID_SIZE):
        for c in range(GRID_SIZE - 3):
            if board[r][c] == board[r][c+1] == board[r][c+2] == board[r][c+3] != EMPTY:
                return board[r][c], [(r,c), (r,c+1), (r,c+2), (r,c+3)]
    for c in range(GRID_SIZE):
        for r in range(GRID_SIZE - 3):
            if board[r][c] == board[r+1][c] == board[r+2][c] == board[r+3][c] != EMPTY:
                return board[r][c], [(r,c), (r+1,c), (r+2,c), (r+3,c)]
    for r in range(GRID_SIZE - 3):
        for c in range(GRID_SIZE - 3):
            if board[r][c] == board[r+1][c+1] == board[r+2][c+2] == board[r+3][c+3] != EMPTY:
                return board[r][c], [(r,c), (r+1,c+1), (r+2,c+2), (r+3,c+3)]
    for r in range(GRID_SIZE - 3):
        for c in range(3, GRID_SIZE):
            if board[r][c] == board[r+1][c-1] == board[r+2][c-2] == board[r+3][c-3] != EMPTY:
                return board[r][c], [(r,c), (r+1,c-1), (r+2,c-2), (r+3,c-3)]
    for r in range(GRID_SIZE):
        for c in range(GRID_SIZE):
            if board[r][c] == EMPTY: return False, []
    return "Tie", []
```

Trong trường hợp hàm đã quét hết bàn cờ mà tìm được chuỗi thắng, hàm `make_move` sẽ quét ra kết quả. Nếu phát hiện có kết quả trả về khác False, hệ thống sẽ lưu lại người thắng, lưu danh sách tọa độ chuỗi 4 quân và kích hoạt biến `self.game_over = True`

```
def make_move(self, row, col): 3 usages
    if self.grid[row][col] == EMPTY and not self.game_over:
        self.grid[row][col] = self.current_player
        winner, cells = check_win_logic(self.grid)
        if winner:
            self.winner = winner
            self.win_cells = cells
            self.game_over = True
        else:
            self.current_player = PLAYER_0 if self.current_player == PLAYER_X else PLAYER_X
        return True
    return False
```

## 6. Thuật toán Minimax đã cài đặt

Trong bài tập này, thuật toán Minimax được xác định là ở Level 1 cho việc tính toán và xác định nước đi.

### 6.1. Cơ chế Min và Max

Thuật toán mô phỏng một cây trò chơi, trong đó hai bên luân phiên tối ưu hóa lợi ích của mình:

- Lượt MAX (Máy tính - O): Thuật toán duyệt qua các nước đi hợp lệ và chọn nước đi mang lại giá trị đánh giá lớn nhất cho AI.

- Lượt MIN (Người chơi - X): Đóng vai trò giả định đối thủ sẽ phản công thông minh nhất, thuật toán tìm và chọn nước đi mang lại giá trị đánh giá nhỏ nhất (bất lợi nhất cho AI). Sau khi đệ quy phân tích, thuật toán Minimax luôn trả về: Nước đi tốt nhất và giá trị đánh giá tương ứng với nước đi đó

## 6.2. Xử lý trạng thái kết thúc và giới hạn độ sâu

Để ngăn chặn việc AI phải tính toán quá nhiều bước đi, quá trình đệ quy của Minimax bị ràng buộc bởi các điều kiện dừng:

- Trạng thái thắng/thua/hòa: Nếu trạng thái đang xét là trạng thái 1 trong 3 trạng thái trên, hàm quét sẽ nhận diện và trả về giá trị tương ứng.

- Đạt giới hạn độ sâu: Thuật toán được cấp một tham số depth. Tham số này giảm đi 1 sau mỗi tầng đệ quy. Khi depth = 0, hàm đệ quy Minimax lập tức dừng lại và gọi hàm đánh giá trạng thái để chấm điểm cục diện bàn cờ chưa kết thúc lúc đó.

## 6.3. Cách thức AI tìm kiếm nước đi qua Depth

Quy trình thực hiện: Chương trình sẽ chạy từ độ sâu 1-3 để tìm kiếm nước đi với giới hạn thời gian là 5s.

- AI sẽ bắt đầu tìm kiếm từ Depth = 1, lưu lại nước đi tốt nhất.
- Sau đó tiếp tục lặn xuống Depth = 2, cập nhật nước đi tốt hơn.
- Quá trình lặp lại cho đến Depth = 3. Nếu trong quá trình tính toán mà vượt quá giới hạn 5s, thuật toán sẽ ngắt ngay lập tức nhánh đệ quy hiện tại và sử dụng nước đi tốt nhất đã tính xong ở độ sâu ngay trước đó để thực hiện.

```
#Thuat toan Minimax + AB
def minimax_standard(board, depth, is_maximizing, start_time, time_limit): 3 usages
    global states_checked
    states_checked += 1

    if time.time() - start_time > time_limit:
        raise TimeoutException()

    if depth == 0: return evaluate_board_for_ai(board, not is_maximizing), None, None

    moves = generate_moves(board)
    if not moves: return evaluate_board_for_ai(board, not is_maximizing), None, None

    best_r, best_c = moves[0]
```

```

if is_maximizing:
    best_score = -math.inf
    for r, c in moves:
        board[r][c] = PLAYER_0
        score, _, _ = minimax_standard(board, depth - 1, is_maximizing: False, start_time, time_limit)
        board[r][c] = EMPTY
        if score > best_score:
            best_score = score
            best_r, best_c = r, c
    return best_score, best_r, best_c
else:
    best_score = math.inf
    for r, c in moves:
        board[r][c] = PLAYER_X
        score, _, _ = minimax_standard(board, depth - 1, is_maximizing: True, start_time, time_limit)
        board[r][c] = EMPTY
        if score < best_score:
            best_score = score
            best_r, best_c = r, c
    return best_score, best_r, best_c

```

## 7. Thuật toán Alpha Beta Pruning đã cài đặt

Mặc dù Minimax đã hoạt động tốt, nhưng với hệ số phân nhánh khổng lồ của cờ Caro, Minimax tỏ ra chậm chạp khi tăng độ sâu, cụ thể là ở mức 3 khi mà thuật toán có thể xét hàng vạn nước đi khác nhau khiến cho việc tính toán mất rất nhiều thời gian. Để giải quyết vấn đề này, Ta tiến lên level 2 – Alpha Beta Pruning. Đây là một bản nâng cấp trực tiếp từ Minimax, giúp loại bỏ các nhánh không cần thiết mà vẫn đảm bảo luôn tìm ra nước đi tối ưu giống Minimax.

- Tham số Alpha và Beta: Thuật toán sử dụng hai giá trị biên để quản lý quá trình tìm kiếm. Alpha đại diện cho giá trị lớn nhất mà Max (Máy tính) có thể đảm bảo đạt được, trong khi Beta đại diện cho giá trị nhỏ nhất mà Min (Người chơi) có thể ép đối thủ phải nhận. Các giá trị này được cập nhật liên tục xuyên suốt quá trình đệ quy qua các tầng của cây tìm kiếm.

- Cơ chế cắt tỉa (Pruning): Trong quá trình duyệt, nếu tại một nhánh con, giá trị Beta trở nên nhỏ hơn hoặc bằng Alpha, hệ thống sẽ ngay lập tức ngừng duyệt các nhánh còn lại của nút đó.

- Tối ưu hóa bằng Move Ordering: Thuật toán Alpha-Beta chỉ phát huy tối đa sức mạnh cắt tỉa nếu nó tìm thấy nước đi tốt sớm nhất có thể. Do đó, hệ thống không duyệt các ô trống một cách ngẫu nhiên mà kết hợp thêm hàm `order_moves`. Hàm này đánh giá sơ bộ và sắp xếp ưu tiên các nước cờ tiềm năng theo thứ tự điểm số từ cao xuống thấp.



```

def minimax_ab(board, depth, is_maximizing, alpha, beta, start_time, time_limit): 3 usages
    global states_checked
    states_checked += 1

    if time.time() - start_time > time_limit:
        raise TimeoutException()

    if depth == 0: return evaluate_board_for_ai(board, not is_maximizing), None, None

    moves = order_moves(board, generate_moves(board), is_maximizing)
    if not moves: return evaluate_board_for_ai(board, not is_maximizing), None, None

    best_r, best_c = moves[0]

    if is_maximizing:
        best_score = -math.inf
        for r, c in moves:
            board[r][c] = PLAYER_O
            score, _, _ = minimax_ab(board, depth - 1, is_maximizing: False, alpha, beta, start_time, time_limit)
            board[r][c] = EMPTY

            if score > alpha: alpha = score
            if score >= beta: return score, r, c

            if score > best_score:
                best_score = score
                best_r, best_c = r, c

        return best_score, best_r, best_c
    else:
        best_score = math.inf
        for r, c in moves:
            board[r][c] = PLAYER_X
            score, _, _ = minimax_ab(board, depth - 1, is_maximizing: True, alpha, beta, start_time, time_limit)
            board[r][c] = EMPTY

            if score < beta: beta = score
            if score <= alpha: return score, r, c
            if score < best_score:
                best_score = score
                best_r, best_c = r, c

        return best_score, best_r, best_c

```

## 8. Hàm đánh giá trạng thái

Khi thuật toán tìm kiếm chạm đến giới hạn độ sâu (depth = 0), ván cờ thường chưa phân định thắng thua. Lúc này, hàm đánh giá trạng thái được sử dụng như một thước đo sức mạnh tổng thể để AI có thể xem được cục diện và đưa ra quyết định.

### 8.1. Cơ chế quét ô

Thay vì đánh giá từng ô đơn lẻ, thuật toán gom các ô cờ thành các danh sách tuyến tính theo 3 hướng: Ngang, Dọc và Chéo. Trên mỗi tuyến, hệ thống đếm liên tục số quân cờ cùng loại đứng sát nhau (count) và nhận diện số lượng chướng ngại vật ở hai đầu (blocks).

## 8.2. Hệ thống đánh giá điểm

| Trạng thái bàn cờ      | Số quân hiện có (Count) | Số quân bị chặn (Block) | Điểm nếu lượt kế là lượt của AI | Điểm nếu lượt kế là lượt của đối thủ (Người hoặc AI) |
|------------------------|-------------------------|-------------------------|---------------------------------|------------------------------------------------------|
| Thắng/Chặn thắng       | 4                       | 0 hoặc 1                | 10000000 ( Biến win_score)      | 10000000                                             |
| 3 quân thoát           | 3                       | 0                       | 1000000                         | 250000                                               |
| 3 quân bị chặn một đầu | 3                       | 1                       | 1000000                         | 200                                                  |
| 2 quân thoát           | 2                       | 1                       | 50000                           | 200                                                  |
| 2 quân bị chặn một đầu | 2                       | 1                       | 10                              | 5                                                    |
| Quân lẻ                | 1                       | Bất kỳ                  | 1                               | 1                                                    |

## 8.3. Phân tích tư duy thông minh của AI qua hàm đánh giá

Hàm đánh giá trên thể hiện sự nhạy bén chiến thuật của AI ở 2 khía cạnh:

- AI xem xét việc lượt kế là lượt của ai để đánh giá điểm tùy vào 2 trường hợp đó. Lấy ví dụ vào trường hợp 3 quân bị chặn 1 đầu

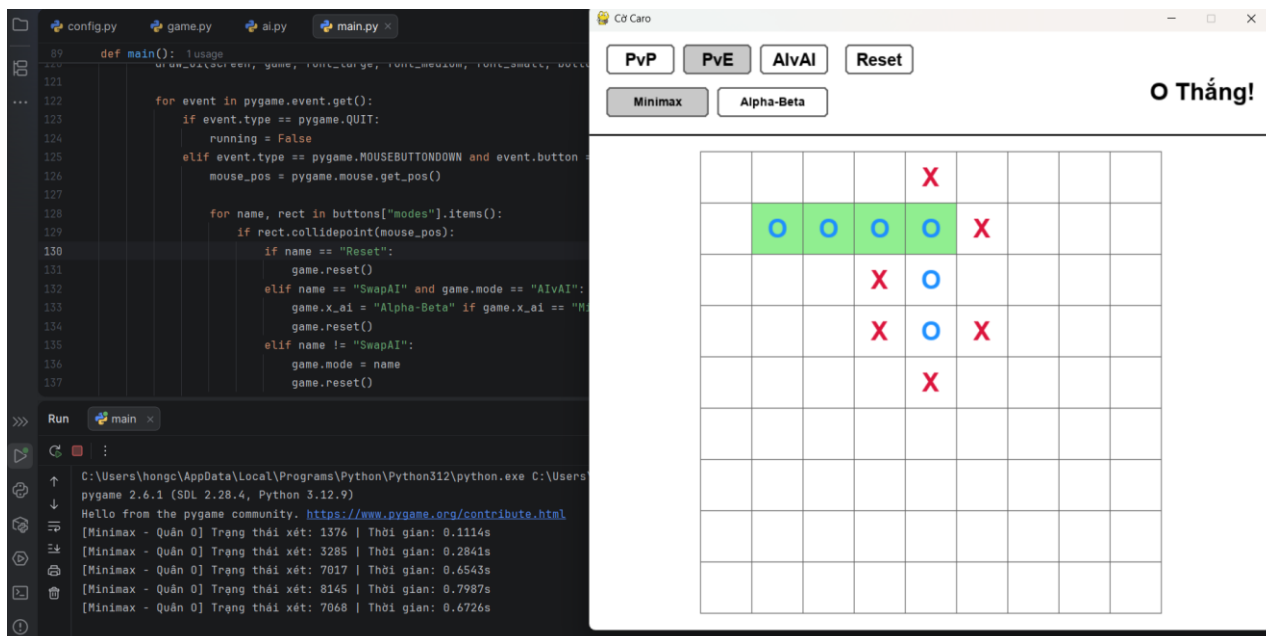
+ Nếu tới lượt AI đánh hệ thống trả về 1.000.000 điểm vì AI chỉ cần đặt 1 quân nữa là đủ 4 quân và chiến thắng.

+ Nếu chưa tới lượt AI, đối thủ chắc chắn sẽ đánh chặn cái đầu hở duy nhất đó. Thế cờ bị phá vỡ dễ dàng nên AI đánh giá nó cực thấp với chỉ 200 điểm.

- Ưu tiên thế cờ thoát: Khi chưa tới lượt AI, thế cờ 3 quân thoát vẫn mang lại điểm số rất cao (250.000 điểm) so với thế 3 quân bị chặn 1 đầu (200 điểm). Lý do là vì đối thủ chỉ có thể chặn 1 đầu, đầu còn lại AI vẫn sẽ dùng để kết thúc ván cờ ở lượt kế tiếp.

## 9. Giao diện của game

\*Em đã xóa đi phần AlvsAI vì có nhiều bug không xử lý được

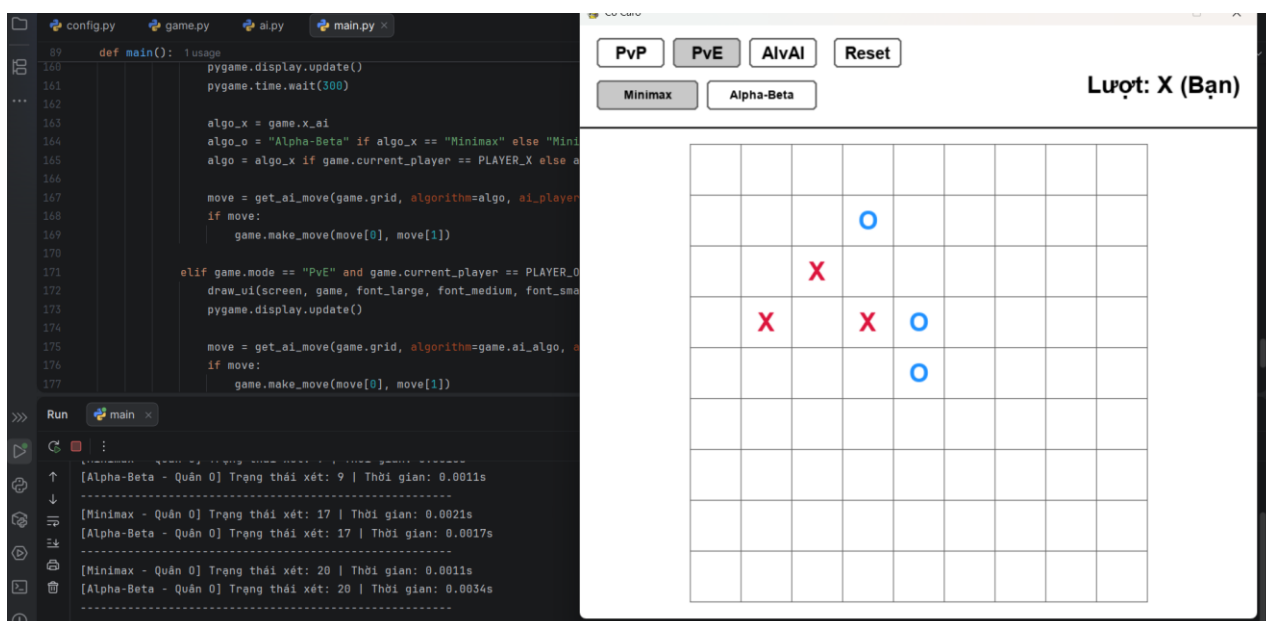


## 10. Bảng kết quả thực nghiệm và nhận xét

\* Để dễ thay đổi Depth và tiện cho việc thực nghiệm, ta thay Depth từ 1 – 3 bằng thay đổi số cho hàng dưới và giả sử thời gian chạy rất cao vì thuật toán Minimax depth 3 chạy rất tốn thời gian. Thực tế thì `depth_limit = 3` và `time_limit = 5s`

```
TIME_LIMIT = 100
depth_limit = 1
```

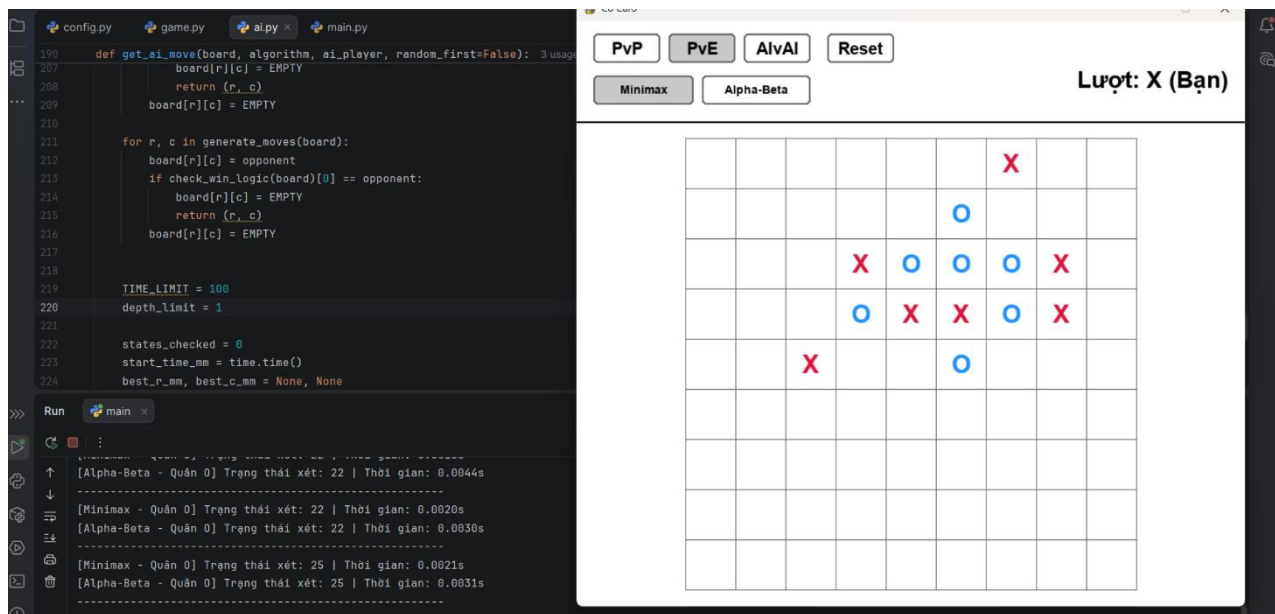
### 10.1. Trạng thái đầu ván



| Độ sâu<br>(Depth) | Tiêu chí đo lường    | Minimax | Alpha-Beta | Hiệu quả cải tiến của Alpha-Beta |
|-------------------|----------------------|---------|------------|----------------------------------|
| <b>Depth = 1</b>  | Số Node trung bình   | 15      | 15         | Bằng nhau                        |
|                   | Thời gian trung bình | 0.0014s | 0.0021s    | Chậm hơn ~1.5 lần                |
| <b>Depth = 2</b>  | Số Node trung bình   | 332     | 80         | Giảm 75.9%                       |
|                   | Thời gian trung bình | 0.0249s | 0.0366s    | Chậm hơn ~1.4 lần                |
| <b>Depth = 3</b>  | Số Node trung bình   | 4890    | 423        | Giảm 91.3%                       |
|                   | Thời gian trung bình | 0.4564s | 0.1557s    | Nhanh hơn ~2.93 lần              |

Dữ liệu thực nghiệm được thu thập tại giai đoạn đầu trận – thời điểm bàn cờ còn rất nhiều ô trống và hệ số phân nhánh đạt cực đại đã khắc họa rõ nét bản chất đánh đổi của thuật toán Alpha-Beta. Ở độ sâu nông nhất (Depth = 1), việc chưa xuất hiện các nhánh đệ quy sâu để cắt tỉa khiến Alpha-Beta tỏ ra chậm chạp hơn Minimax 1.5 lần, do thuật toán phải gánh chịu toàn bộ chi phí thời gian từ hàm Move Ordering khi phải chấm điểm cho hàng loạt ô trống. Sang Depth = 2, dù Alpha-Beta đã bắt đầu loại bỏ được 75.9% số trạng thái dư thừa, nhưng thời gian chạy vẫn nhỉnh hơn đôi chút (0.0366s so với 0.0249s) vì lượng thời gian tiết kiệm được ở các nhánh nông chưa đủ bù đắp cho khối lượng tính toán khổng lồ của khâu xử lý. Tuy nhiên, ta có thể thấy rõ cải thiện ở Depth = 3: Minimax chật vật duyệt tới 4.890 Node. Trong khi đó, Alpha-Beta nhanh chóng tìm ra các điểm ngắt, triệt tiêu tới 91.3% cây tìm kiếm (chỉ còn 423 Node). Lợi ích khổng lồ từ việc tránh được hàng ngàn nhánh đệ quy sâu đã hoàn toàn áp đảo chi phí tiền xử lý ban đầu, giúp Alpha-Beta bứt tốc và chạy nhanh gấp gần 3 lần Minimax (0.1557s so với 0.4564s).

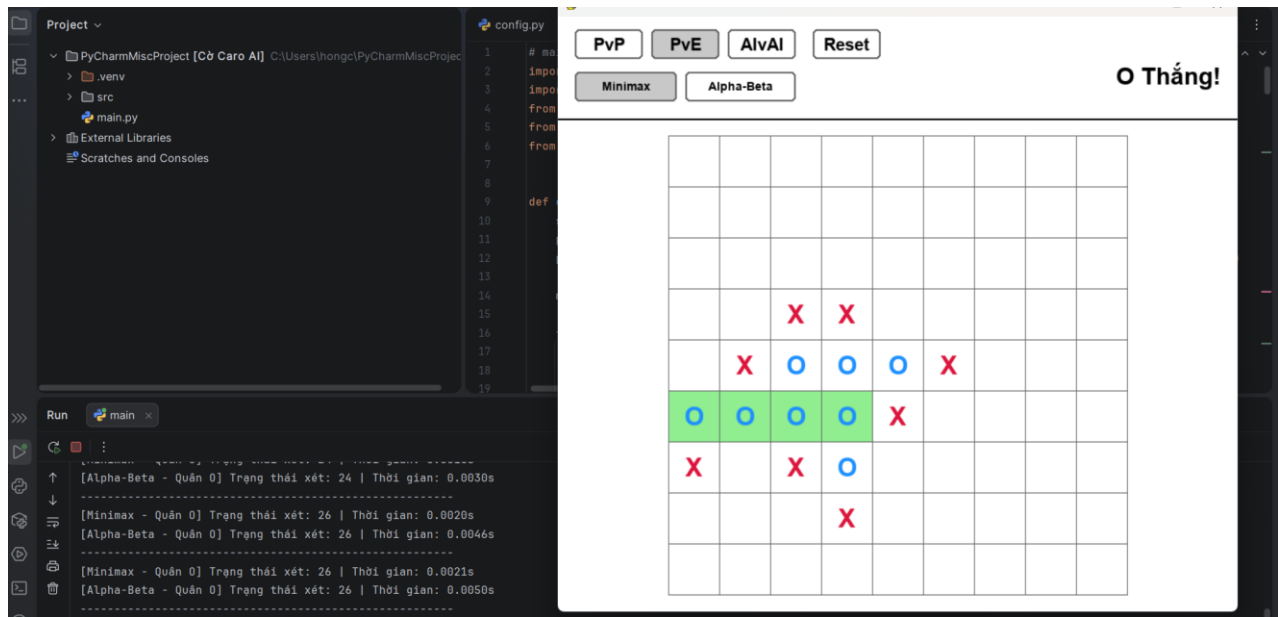
## 10.2. Trạng thái giữa trận



| Độ sâu (Depth) | Tiêu chí đo lường    | Minimax | Alpha-Beta | Hiệu quả cải tiến của Alpha-Beta |
|----------------|----------------------|---------|------------|----------------------------------|
| Depth = 1      | Số Node trung bình   | ~ 17    | ~ 17       | Bằng nhau                        |
|                | Thời gian trung bình | 0.0012s | 0.0024s    | Chậm hơn ~2.0 lần                |
| Depth = 2      | Số Node trung bình   | 418     | 91         | Giảm 78.2%                       |
|                | Thời gian trung bình | 0.0356s | 0.0541s    | Chậm hơn ~1.5 lần                |
| Depth = 3      | Số Node trung bình   | 9048    | 607        | Giảm 93.3%                       |
|                | Thời gian trung bình | 0.8950s | 0.2555s    | Nhanh hơn ~ 3.5 lần              |

Dữ liệu thực nghiệm tại giai đoạn giữa trận càng cho thấy rõ bài toán đánh đổi của thuật toán Alpha-Beta. Đặc thù của giai đoạn này là bàn cờ xuất hiện nhiều cụm quân đan xen chằng chịt, khiến khối lượng tính toán cho hàm đánh giá và Move Ordering tăng rất cao. Ở các độ sâu nông (Depth = 1 và 2), mặc dù Alpha-Beta đã bộc lộ khả năng cắt tỉa ấn tượng (loại bỏ tới 78.2% số Node ở Depth 2), thời gian thực thi của nó vẫn chậm hơn Minimax từ 1.5 đến 2 lần. Nguyên nhân là do lượng thời gian tiết kiệm được từ việc cắt vài nhánh nông chưa đủ để bù đắp cho chi phí xử lý quá nặng khi phải chấm điểm hàng loạt thế cờ phức tạp. Tuy nhiên, cục diện hoàn toàn đảo chiều khi hệ thống sang Depth = 3. Nhờ cơ chế sắp xếp nước đi nhạy bén, Alpha-Beta ưu tiên duyệt các nước Thắng hoặc Chặn thắng trước, từ đó tạo ra các điểm ngắt giúp triệt tiêu tới 93.3% không gian tìm kiếm (từ hơn 9.000 Node xuống vồn vện 607 Node). Sự chính xác trong việc chọn lọc nhánh duyệt đã mang lại lợi ích thời gian khổng lồ, áp đảo hoàn toàn chi phí và giúp Alpha-Beta đạt tốc độ nhanh gấp 3.5 lần so với Minimax (0.2555s so với 0.8950s).

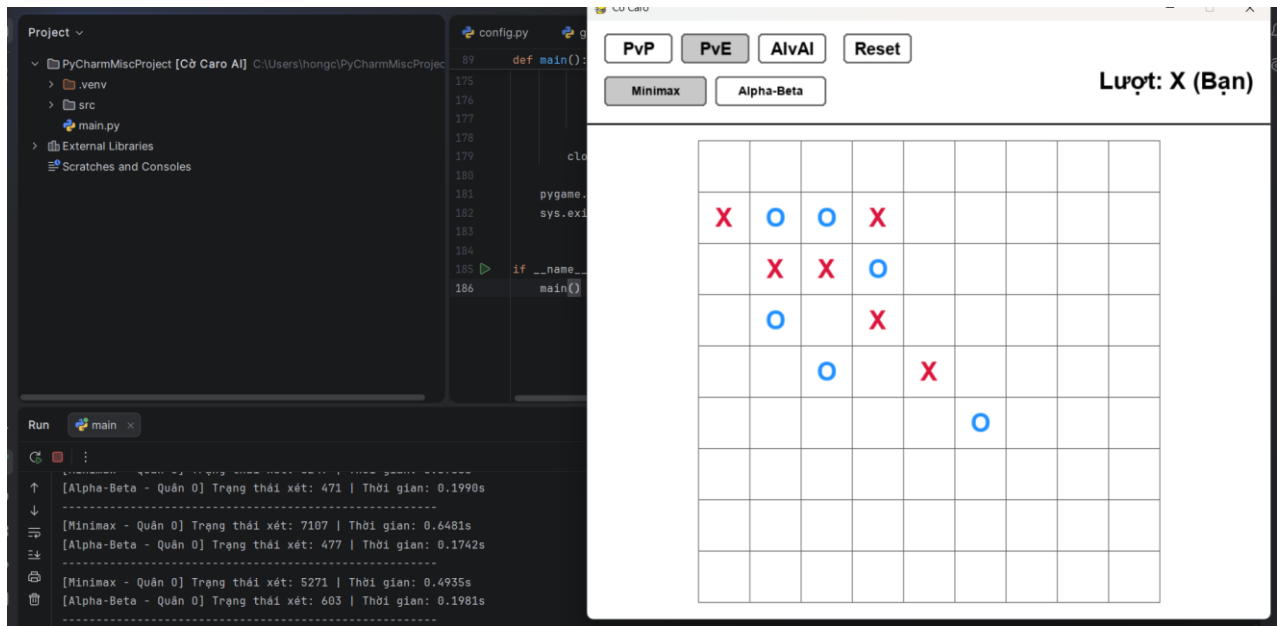
### 10.3. Trạng thái máy có thể thắng ngay



| Độ sâu (Depth) | Tiêu chí đo lường    | Minimax | Alpha-Beta | Hiệu quả cải tiến của Alpha-Beta |
|----------------|----------------------|---------|------------|----------------------------------|
| Depth = 1      | Số Node trung bình   | ~ 18    | ~ 18       | Bằng nhau                        |
|                | Thời gian trung bình | 0.0013s | 0.0029s    | Chậm hơn ~2.2 lần                |
| Depth = 2      | Số Node trung bình   | 437     | 80         | Giảm 81.6%                       |
|                | Thời gian trung bình | 0.0366s | 0.0491s    | Chậm hơn ~1.3 lần                |
| Depth = 3      | Số Node trung bình   | 8788    | 566        | Giảm 93.5%                       |
|                | Thời gian trung bình | 0.8518s | 0.2398s    | Nhanh hơn ~3.55 lần              |

Ở trạng thái AI có thể hạ quân thắng ngay, thực nghiệm ghi nhận số Node và thời gian duyệt của cả hai thuật toán đều tiến sát về 0 (duyet 2 trạng thái trong 0.0001s). Hiện tượng này xảy ra nhờ cơ chế kiểm tra điều kiện thắng - thua ưu tiên được tích hợp trong thuật toán của AI

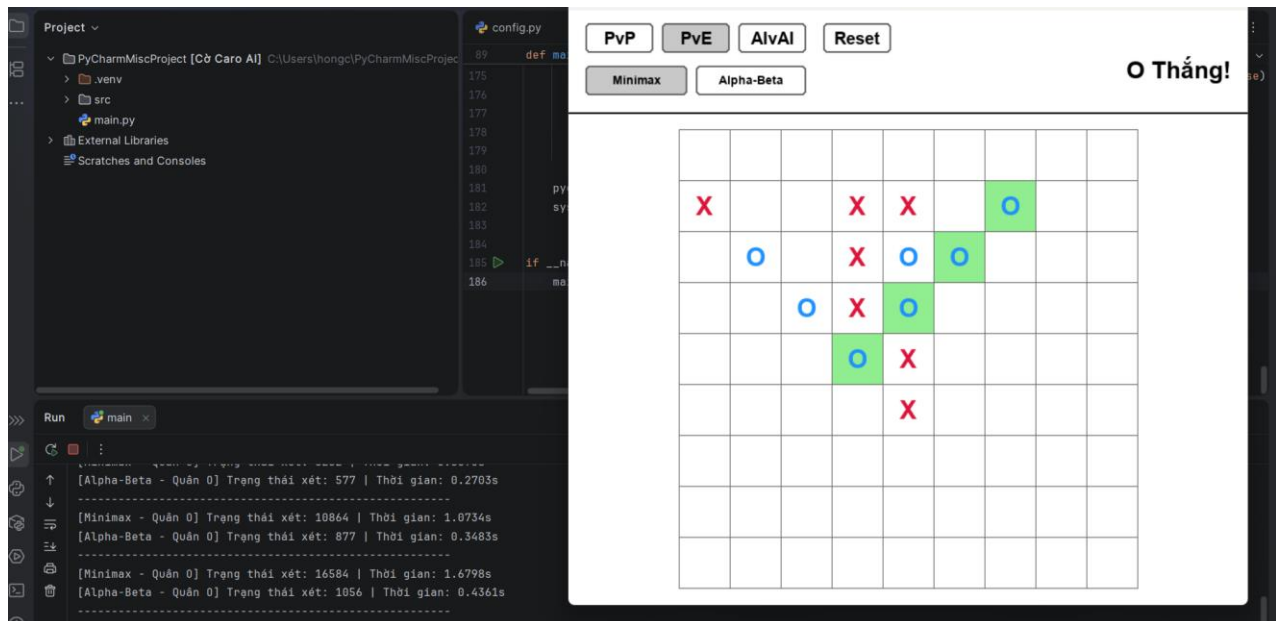
## 10.4. Trạng thái máy phải chặn thắng



| Độ sâu (Depth) | Tiêu chí đo lường    | Minimax | Alpha-Beta | Hiệu quả cải tiến của Alpha-Beta |
|----------------|----------------------|---------|------------|----------------------------------|
| Depth = 1      | Số Node trung bình   | ~ 16    | ~ 16       | Bằng nhau                        |
|                | Thời gian trung bình | 0.0016s | 0.0022s    | Chậm hơn ~1.38 lần               |
| Depth = 2      | Số Node trung bình   | 562     | 93         | Giảm 83.5%                       |
|                | Thời gian trung bình | 0.0342s | 0.0612s    | Chậm hơn ~1.79 lần               |
| Depth = 3      | Số Node trung bình   | 9582    | 709        | Giảm 92.6%                       |
|                | Thời gian trung bình | 0.9570s | 0.2431s    | Nhanh hơn ~3.94 lần              |

Ở trạng thái AI có thể hạ quân thắng ngay, thực nghiệm ghi nhận số Node và thời gian duyệt của cả hai thuật toán đều tiến sát về 0 (duyet 2 trạng thái trong 0.0001s). Hệ thống lập tức bắt được tín hiệu nguy hiểm, ngay lập tức ngắt toàn bộ tiến trình đệ quy sâu và trả về tọa độ phòng ngự trong chớp mắt. Cơ chế này đảm bảo AI không bị sa đà vào việc tính toán các nhánh đệ quy vô nghĩa khi sự sống còn của ván đấu đang bị đe dọa trực tiếp.

## 10.5. Trạng thái máy phải lựa chọn giữa Thắng hoặc Chặn Thắng



| Độ sâu (Depth) | Tiêu chí đo lường    | Minimax | Alpha-Beta | Hiệu quả cải tiến của Alpha-Beta |
|----------------|----------------------|---------|------------|----------------------------------|
| Depth = 1      | Số Node trung bình   | ~ 16    | ~ 16       | Bằng nhau                        |
|                | Thời gian trung bình | 0.0016s | 0.0029s    | Chậm hơn ~1.81 lần               |
| Depth = 2      | Số Node trung bình   | 836     | 105        | Giảm ~87.4%                      |
|                | Thời gian trung bình | 0.0567s | 0.0713s    | Chậm hơn ~1.26 lần               |
| Depth = 3      | Số Node trung bình   | 13942   | 901        | Giảm ~93.5%                      |
|                | Thời gian trung bình | 1.2631s | 0.3487s    | Nhanh hơn ~3.62 lần              |

Càng đánh về sau ta càng thấy rõ hơn độ linh hoạt của thuật toán Alpha-Beta so với Minimax khi ở Depth = 3: Đặc thù của tình huống này là bàn cờ đã tiến về giai đoạn cuối game, cả AI và người chơi đều đang sở hữu các thế cờ nguy hiểm, đẩy độ phức tạp của không gian tìm kiếm lên mức cực đại. Khi hệ thống phải quét ở độ sâu lớn, Minimax lập tức bộc lộ sự quá tải khi phải duyệt tới gần 14.000 Node và mất hơn 1.26 giây để phản hồi. Ngược lại, nhờ cơ chế Move Ordering, Alpha-Beta đã chặt đứt các nhánh đệ quy vô ích, giúp loại bỏ tới 93.5% số Node dư thừa và nới rộng khoảng cách tốc độ lên nhanh gấp 3.6 lần so với Minimax.



# 11. Đánh giá và thảo luận tổng quan

## 11.1. Alpha-Beta có chọn cùng nước đi với Minimax không?

Về bản chất toán học, Alpha-Beta Pruning không phải là một thuật toán xấp xỉ mà là một kỹ thuật tối ưu hóa của Minimax. Do đó, nước đi do Alpha-Beta chọn có chất lượng chiến thuật hoàn toàn tương đương với Minimax. Thuật toán chỉ cắt bỏ những nhánh chắc chắn đem lại kết quả tồi hơn, chứ không bao giờ cắt nhầm nhánh chứa nước đi tối ưu. Nếu như trong trường hợp Depth cao hơn như 3,4 mà giới hạn thời gian duyệt lại, Minimax sẽ chọn nước đi kém tối ưu hơn so với Alpha-Beta do phải duyệt quá nhiều ô mà không đủ thời gian

## 11.2. Alpha-Beta giảm được bao nhiêu trạng thái so với Minimax?

Dựa trên dữ liệu thực nghiệm đã thu thập (tại Depth = 3), Alpha-Beta cho thấy khả năng cắt tỉa không gian tìm kiếm cực kỳ tốt. Tùy thuộc vào giai đoạn của ván cờ, Alpha-Beta giúp giảm từ 91.3% lên tới 93.5% tổng số lượng trạng thái phải duyệt so với Minimax. Điều này tương đương với việc tránh được hàng chục nghìn nhánh đệ quy vô ích, khẳng định vai trò của Alpha-Beta trong các bài toán có hệ số phân nhánh khổng lồ như cờ Caro.

## 11.3. Thời gian chạy thay đổi như thế nào khi tăng độ sâu?

Thời gian chạy của hệ thống tăng theo cấp số nhân. Cụ thể:

- Từ Depth 1 lên Depth 2: Không gian trạng thái mở rộng, thời gian thực thi của cả hai thuật toán tăng nhẹ nhưng vẫn nằm trong ngưỡng thấp.
- Từ Depth 2 lên Depth 3: Thời gian chạy tăng đột biến. Minimax bị quá tải nghiêm trọng, có thể mất từ 1.0s đến gần 2.0s để ra quyết định ở giữa trận. Trong khi đó, Alpha-Beta nhờ cắt tỉa thành công nên giữ được mức tăng thời gian tương đối tuyến tính, duy trì tốc độ phản hồi ổn định ở mức ~0.25s - 0.35s.

## 11.4. Độ sâu tìm kiếm ảnh hưởng thế nào đến chất lượng nước đi?

Độ sâu tìm kiếm (Depth) tỷ lệ thuận trực tiếp với tầm nhìn chiến thuật của AI:

- Depth = 1: AI chỉ nhìn thấy cái lợi trước mắt, tức AI ưu tiên ăn ô có điểm cao nhất hiện tại mà không biết tính toán hệ quả lượt sau.
- Depth = 2: AI đã biết phân tích một nhịp phản công của đối thủ, hình thành được tư duy phòng ngự cơ bản.
- Depth = 3: Đây là ngưỡng AI bắt đầu bộc lộ sự thông minh. Máy không chỉ biết phòng ngự mà còn biết cách tạo ra các thế cờ đôi và lừa người chơi

## 11.5. Hàm đánh giá có ưu điểm gì và còn hạn chế gì?

- Ưu điểm: Hàm Heuristic hiện tại có ưu điểm là tốc độ tính toán cực nhanh, tốn rất ít tài nguyên bộ nhớ. Nó thiết lập được các trọng số khắt khe để phân biệt rõ ràng giữa thế cờ "Sắp thắng" (1 triệu điểm) và "Thắng luôn" (10 triệu điểm), giúp AI có phản xạ chớp thời cơ và phòng ngự rất tốt.

- Hạn chế: Hàm đánh giá vẫn còn sơ sài. AI chỉ chăm chăm nhìn vào các đường (ngang, dọc, chéo) để đếm quân mà thiếu đi khả năng nhìn tổng quát cả bàn cờ. Nó không hiểu được khái niệm kiểm soát trung tâm bàn cờ là lợi thế lâu dài, dẫn đến việc đôi khi AI rải quân ở các góc khuất một cách lãng phí.

## **11.6. Trường hợp nào AI chơi tốt và trường hợp nào AI chọn nước đi chưa hợp lý?**

- Trường hợp AI chơi tốt: Ở giữa trận và cuối trận, khi các thế cờ nguy hiểm xuất hiện liên tục. Thuật toán AI có khả năng bắt lỗi tốt, chớp cơ hội hạ quân chiến thắng hoặc chặn đứng chuỗi 3 của người chơi trong thời gian rất ngắn ( $< 0.0002s$ ).

- Trường hợp AI chơi chưa hợp lý: Ở những nước đi đầu tiên, khi bàn cờ trống rỗng, các ô vuông đều mang lại mức điểm Heuristic na ná nhau. Máy tính không có định hướng chiến lược dài hạn nên thường đi những nước ngẫu nhiên hoặc rải quân rời rạc thay vì tập trung xây dựng thế trận liên kết.

## **11.7. Nếu cải tiến tiếp, sinh viên sẽ cải tiến phần nào?**

Nếu có thêm thời gian phát triển dự án, hệ thống có thể được nâng cấp ở 3 khía cạnh sau:

- Nâng cấp hàm đánh giá: Hàm tính điểm hiện tại đang đánh giá các đường dọc/ngang/chéo một cách độc lập nên đôi khi vẫn bị sập bẫy các thế cờ nước đôi của người chơi. Cần cải tiến thuật toán để AI có khả năng nhận diện và phân tích sự liên kết không gian giữa nhiều đường cờ cùng lúc.

- Cải thiện trải nghiệm người dùng: Bổ sung các tính năng linh hoạt cho giao diện như: cho phép người chơi tùy chỉnh kích thước bàn cờ; thêm tùy chọn chuyển đổi giữa luật thắng 4 quân hiện tại sang luật 5 quân

- Thêm chế độ AI vs AI để cho thuật toán tự chơi với nhau: Phần này nhóm em có ý tưởng làm nhưng rất lười lười nên đã phải bỏ đi.

# **12. Ưu điểm và nhược điểm**

## **12.1. Ưu điểm**

- Giao diện game trực quan: Sử dụng thư viện Pygame gọn nhẹ khiến cho UI của game dễ nhìn và dễ chơi, đồng thời đánh dấu màu của 4 ô thắng cuộc khiến người chơi dễ quan sát

- Đảm bảo tính tối ưu trong giới hạn: Bản chất của thuật toán Minimax kết hợp Alpha-Beta đảm bảo AI luôn chọn được nước đi tốt nhất dựa trên hàm đánh giá trong phạm vi độ sâu (Depth) được thiết lập.

- Hiệu năng cắt tỉa vượt trội: Việc áp dụng thành công thuật toán Alpha-Beta Pruning kết hợp hàm Move Ordering đã giải quyết xuất sắc việc xét quá nhiều ô của thuật toán. Hệ thống giảm tải được hơn 93% lượng trạng thái phải duyệt ở những pha cờ phức tạp, giúp AI duy trì tốc độ phản hồi mượt mà theo thời gian thực.

- Phản xạ khẩn cấp nhảy bén: Nhờ tích hợp cơ chế ngắt quãng ngay từ đầu chu trình, AI có thể lập tức chặn đứng các đường cờ nguy hiểm của người chơi hoặc tự động hạ quân chốt hạ trong tích tắc ( $< 0.0002s$ ) mà không lãng phí tài nguyên tính toán.

## **12.2. Nhược điểm**

- Thiếu tầm nhìn chiến lược toàn bàn cờ: Hàm Heuristic hiện tại hoạt động vẫn còn đơn giản - chỉ đếm số lượng quân trên từng đường đơn lẻ. Hệ thống chưa có khả năng đánh giá mức độ kiểm soát bàn cờ tổng thể, dẫn đến việc đôi khi AI dễ bị mắc lừa bởi các thế nước đôi.

- Giai đoạn khai cuộc vẫn còn kém: AI khởi đầu ván đấu vẫn còn khá máy móc và chưa biết cách chủ động chiếm lĩnh các vị trí đặc địa.

## **13. Tài liệu tham khảo**

Tham khảo từ dự án gomokuAI:

- Cách biểu diễn bàn cờ dưới dạng mảng 2 chiều và cách làm hàm kiểm tra điều kiện kết thúc bàn cờ (Thắng/Hòa/Thua)

- Hàm đánh giá Heuristic: Cách làm hàm đánh giá để làm điểm cho con AI đi thông minh hơn

Tham khảo từ dự án Caro\_AI:

- Cách tùy chỉnh trọng số cho hàm đánh giá để giúp AI không bị nhiễu giá trị

- Cách làm hàm nhận biết nguy hiểm: Logic bắt lỗi tức thời giúp AI có thể chốt hạ hoặc chặn đứng đối thủ trong chưa tới 0.0002 giây, tối ưu hóa triệt để hiệu năng tại các pha cờ nguy hiểm.

Tham khảo ngoài: [https://www.youtube.com/watch?v=7r2vupvjE\\_U](https://www.youtube.com/watch?v=7r2vupvjE_U) và <https://www.youtube.com/watch?v=LbTu0rwikwg&t=336s>:

- Tư duy biểu diễn bàn cờ và làm thuật toán Minimax và Alpha-Beta cho bài toán Cờ Caro